09.05.'23

Eser. 5.10

(a) Siano $Y_1, \dots, Y_n$ delle v.a. di Poisson di parametro 1. Allora (va pensato come $\frac{\#\text{palline}}{\#\text{urne}}$)

$$\mathbb{P}(Y_1 = 1, \dots, Y_n = 1) \overset{\text{indipendenza e stesso distrib.}}{=} \mathbb{P}(Y_1 = 1)^n$$
$$= (e^{-1})^n$$
$$= e^{-n}.$$

(b) Supponiamo di lanciare le palline una alla volta. La probabilità che la seconda cada in un'urna differente dalla prima è $\frac{n-1}{n}$. Che la terza cada in un'urna differente dalle prime due è $\frac{n-2}{n}$, e così via. Quindi, se X_i indica il numero di palline nell'urna $i$esima

$$\mathbb{P}(X_1 = 1, \dots, X_n = 1) = \frac{n-1}{n} \cdot \frac{n-2}{n} \cdot \frac{n-3}{n} \cdot \dots \cdot \frac{2}{n} \cdot \frac{1}{n}$$

(per l'ultima pallina rimane una sola urna a disposizione)

$$= \frac{(n-1)!}{n^{n-1}}$$
$$= \frac{n!}{n^n}$$

(c) Se $Z \sim \text{Poi}(n)$ si ha

$$\mathbb{P}(Z = n) = e^{-n} \frac{n^n}{n!}$$

che è proprio il quoziente delle due

probabilità trovate nei punti (a) e (b).

Questa relazione poteva essere desunta dal Teorema 5.6 che ci dice

$$\mathbb{P}(X_1=1,\dots,X_n=1) = \mathbb{P}\left(Y_1=1,\dots,Y_n=1 \,\middle|\, \hat{\sum_{j=1}} Y_j = n\right)$$

$$= \frac{\mathbb{P}\left(\{Y_1=1,\dots,Y_n=1\} \cap \hat{\sum_{j=1}^{n}} Y_j = n\right)}{\mathbb{P}\left(\hat{\sum_{j=1}^{n}} Y_j = n\right)}$$

$$= \frac{\mathbb{P}(Y_1=1,\dots,Y_n=1)}{\mathbb{P}\left(\hat{\sum_{j=1}^{n}} Y_j = n\right)}$$

e notando che la somma $\hat{\sum_{j=1}} Y_j$ di n Poisson indipendenti di parametro 1 è a sua volta una v.a. di Poisson di parametro n.

CAPITOLO 7 : CATENE DI MARKOV & PASSEGGIATE ALEATORIE

Un <u>processo stocastico</u> $(X_t)_{t\in T}$ è una collezione di v.a. indicizzate da un parametro t, che spesso può essere pensato come il tempo (T può essere sia continuo, ad esempio $T=\mathbb{R}_+$, che discreto, ad esempio $T=\mathbb{N}_0$ ↑ naturali $\cup\{0\}$). $(X_t)_{t\in T}$ descrive in questo caso il variare di una certa quantità col trascorrere del tempo (pensate ad esempio X_t = "prezzo delle azioni Microsoft al giorno t").

X_t è detto <u>stato</u> del processo al tempo t e può anch'esso prendere valori nel continuo ($X_t\in\mathbb{R}^d$ ad esempio) o nel discreto ($X_t\in V$ con V i vertici di un grafo, ad esempio).

Le <u>catene di Markov</u> sono processi stocastici a tempo discreto (spesso anche a spazio discreto) speciali: il valore al tempo t, X_t, dipende solo da X_{t-1} e NON dal resto della storia

passata del processo $X_0, X_1, \dots, X_{t-2}$:

Def. 7.1

Un processo stocastico a tempo discreto $(X_t)_{t \in \mathbb{N}_0}$ è detto <u>catena di Markov</u> (omogenea) se

$$\mathbb{P}(X_t = j \mid X_{t-1} = i, X_{t-2} = i_{t-2}, \dots, X_0 = i_0) = \quad (*)$$

$$= \mathbb{P}(X_t = j \mid X_{t-1} = i) =: P_{ij}$$

per ogni valore di $i, j, i_0, \dots, i_{t-2}$.

La proprietà (*) è detta proprietà di Markov.

<u>Oss.</u> La (*) NON ci dice che X_t è indipendente da $X_0, \dots, X_{t-2}$, ma che lo diventa se la si condiziona ad X_{t-2}.

Se assumiamo che $(X_t)_{t \in \mathbb{N}_0}$ assume valori in un insieme finito $V = \{0, 1, \dots, n\}$ (o infinito numerabile $V = \{0, 1, 2, \dots\}$), la probabilità P_{ij} che appare in (*) è

la probabilità che il processo "salti" da i a j in un passo.

Def. Possiamo raccogliere tutte queste probabilità nella matrice

$$P = \begin{pmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{pmatrix}$$

detta <u>matrice di transizione</u> della catena.

Oss. Si ha che le somme sulle righe valgono

$$\sum_{j \in V} P_{ij} = 1 \qquad \forall i \in V.$$

La matrice P è utile per calcolare la distribuzione della posizione della catena ad un tempo t. Sia $p_i(t) := \mathbb{P}(X_t = i)$ la probabilità che la catena sia in $i \in V$ al tempo t. Sia

$\bar{p}(t) = (p_0(t), p_1(t), \dots p_{|V|}(t))$ il vettore-riga che descrive la distribuzione di X_t. Allora, dalla formula delle probabilità totali:

$$p_j(t) = \sum_{i \in V} p_i(t-1)\, P_{ij}$$

poiché

$$\mathbb{P}(X_t = j) = \sum_{i \in V} \mathbb{P}(X_t = j \mid X_{t-1} = i)\, \mathbb{P}(X_{t-1} = i).$$

In forma matriciale (considerando cioè contemporaneamente tutti gli indici) si ha che

$$\bar{p}(t) = \bar{p}(t-1)\, P.$$

Possiamo anche calcolare, ad esempio, la probabilità che si arrivi in j dopo due passi partendo da i al tempo t:

$$\mathbb{P}(X_{t+2} = j \mid X_t = i) = \sum_{\ell \in V} \mathbb{P}(X_{t+2} = j, X_{t+1} = \ell \mid X_t = i)$$

$$= \sum_{\ell \in V} \mathbb{P}(X_{t+2} = j \mid X_{t+1} = \ell, X_t = i)\, \mathbb{P}(X_{t+1} = \ell \mid X_t = i)$$

$$= \sum_{\ell \in V} P_{\ell j}\, P_{i\ell}$$

$$= (P^2)_{ij}.$$

Generalizzando ad m passi si ha

$$P^m_{ij} := (P^m)_{ij} = \mathbb{P}(X_{t+m} = j \mid X_t = i)$$

con

$$P^m_{ij} = \sum_{l \in V} P_{il} P^{m-1}_{lj}$$

La matrice di transizione ad m passi

$$P^m = \begin{pmatrix} P^m_{11} & P^m_{12} & \cdots & P^m_{1n} \\ \vdots & \ddots & \ddots & \vdots \\ P^m_{n1} & \cdots & \cdots & P^m_{nn} \end{pmatrix}$$

è perciò tale che

$$P^m = P \cdot P^{m-1} = \underbrace{P \cdot \ldots \cdot P}_{m\text{-volte}}$$

e $\forall t \in \mathbb{N}_0$ e $m \in \mathbb{N}$

$$\bar{p}(t+m) = \bar{p}(t) P^m .$$

Un altro utile modo di pensare una catena di Markov è pensarla come un grafo diretto pesato $G = (V, E, w)$. I suoi vertici V sono gli stati della catena. L'arco (diretto)

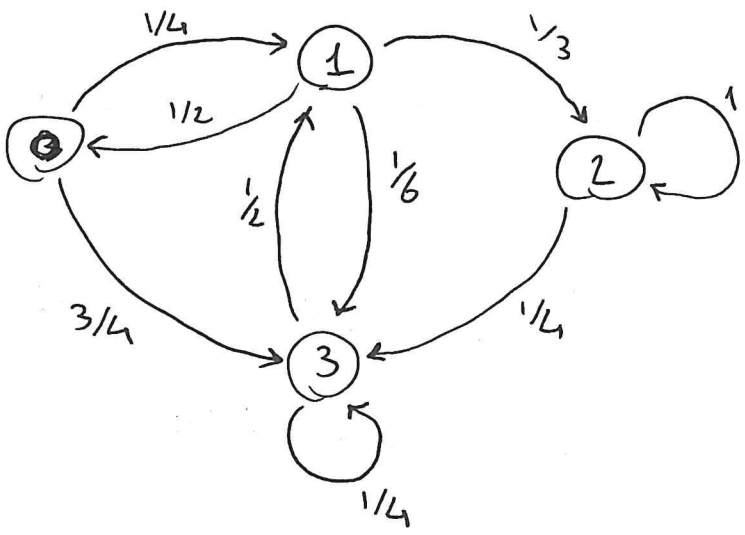
(i,j) è presente se e solo se $P_{ij} > 0$, nel quale caso gli si associa il peso $w_{ij} = P_{ij}$. Sono permessi self-loops (quando $P_{ii} > 0$). Per definizione i pesi soddisfano

$$\sum_{j \in V} w_{ij} = 1 \qquad \forall i \in V.$$

Una sequenza di stati visitati dal processo è rappresentata da un cammino diretto sul grafo. La probabilità che il processo segua un dato cammino è il prodotto dei pesi degli archi utilizzati dal cammino.

ex. La seguente matrice ed il seguente grafo rappresentano la stessa catena di Markov:

$$P = \begin{pmatrix} 0 & 1/4 & 0 & 3/4 \\ 1/2 & 0 & 1/3 & 1/6 \\ 0 & 0 & 1 & 0 \\ 0 & 1/2 & 1/4 & 1/4 \end{pmatrix}$$



Calcoliamo con le due rappresentazioni la probabilità di andare da 0 a 3 in esattamente tre passi.

- Per il grafo consideriamo tutti i cammini che vadano da 0 a 3 in tre passi:

$0 \to 1 \to 0 \to 3$ $\quad 0 \to 1 \to 3 \to 3$ $\quad 0 \to 3 \to 1 \to 3$

$0 \to 3 \to 3 \to 3$.

I cammini hanno probabilità rispettivamente $3/32$, $\frac{1}{96}$, $\frac{1}{16}$, $\frac{3}{64}$. La somma è $\frac{41}{192}$

- Per la matrice, calcoliamo la terza potenza

$$P^3 = \begin{pmatrix} \frac{3}{16} & 7/48 & 29/64 & 41/192 \\ 5/48 & 5/24 & \frac{79}{144} & 5/36 \\ 0 & 0 & 1 & 0 \\ 1/16 & 13/96 & \frac{107}{192} & \frac{47}{192} \end{pmatrix}$$

e notiamo che concordemente $P^3_{03} = \frac{41}{192}$.

Oss: La matrice ci dice anche, ad esempio, la probabilità di essere in 3 dopo tre

passi se cominciamo da un vertice a caso: essa varrà $\frac{43}{288}$ poiché

$$\left(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right) P^3 = \left(\frac{17}{192}, \frac{47}{384}, \frac{737}{1152}, \frac{43}{288}\right).$$

11.05.'23

7.1.1 Applicazione: un algoritmo aleatorio per il problema 2-SAT

Consideriamo problemi di soddisfacibilità Booleana, o SAT.

In una formula logica, un <u>letterale</u> è una variabile Booleana ("vero" o "falso") oppure una sua negazione. Per una variabile x, indichiamo con $\bar{x}$ la sua negazione.

<u>Def.</u> Un <u>problema SAT</u>, o <u>formula SAT</u>, è un'espressione logica data dalla congiunzione (AND) di un insieme di clausole, dove ciascuna clausola è la disgiunzione (OR)

di letterali.

ex.

Date le variabili $x_1, x_2, x_3, x_4 \in \{\text{vero}, \text{falso}\}$ la seguente è una formula SAT:

$$\underbrace{(x_1 \vee \bar{x}_2 \vee \bar{x}_3)}_{\text{clausola}} \wedge (\bar{x}_1 \vee \bar{x}_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (x_4 \vee \bar{x}_3) \wedge (x_4 \vee \bar{x}_1$$

(OR dentro la clausola; AND tra le clausole)

Una soluzione di un problema SAT è un'assegnazione di valori (vero o falso) delle variabili che rende l'espressione vera. Cioè, tale che in ogni clausola ci sia almeno un letterale vero.

ex.

Tornando all'esempio precedente

$x_1 = \text{vero}$ $x_2 = \text{falso}$ $x_3 = \text{falso}$ $x_4 = \text{vero}$

è una soluzione di quello specifico problema SAT.

Nota

In generale determinare se un problema SAT ammette almeno una soluzione è un problema NP-hard.

Analizziamo ora un algoritmo aleatorio per trovare la soluzione di un sotto-problema più semplice, quello del 2-SAT, in cui ogni clausola ha solo due letterali.

ex. Un problema di tipo 2-SAT è il seguente:

$$(x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_3) \wedge (x_1 \vee x_2) \wedge (x_4 \vee \bar{x}_3) \wedge (x_4 \vee \bar{x}_1)$$

L'idea per trovare una soluzione del problema 2-SAT è il seguente: si parte da una qualsiasi assegnazione delle variabili. Poi, ad ogni passo, si sceglie una clausola falsa e si cambia il valore di una delle due varia-

bili in essa contenute

Algoritmo 2-SAT

Input: Un problema 2-SAT con n variabili

Output: Una soluzione del problema o fallimento

1. Inizializzare con una qualsiasi assegnazione delle variabili (ad ex., tutte vere)
2. Per $2mn^2$ volte ripetere: ($m \in \mathbb{N}$ da scegliere)
   (a) Scegliere una qualsiasi clausola non soddisfatta (cioè, falsa)
   (b) Scegliere una delle due variabili della clausola a caso (cioè, con proba $\frac{1}{2}$) e cambiarne il valore.
   (c) Se tutte le clausole sono soddisfatte, terminare il ciclo.
3. Se è stata trovata una soluzione, restituirla in output. Altrimenti, fallimento.

Oss. $m \in \mathbb{N}$ è un intero che determina la probabilità di successo dell'algoritmo.

ex.

Nell'esempio precedente, se cominciamo da $x_1 = x_2 = x_3 = x_4 =$ falso, la clausola $(x_1 \vee x_2)$ non è soddisfatta. L'algoritmo potrebbe allora scegliere questa clausola e cambiare una tra x_1 e x_2, ad esempio potrebbe porre $x_1 =$ vero. In questo caso la clausola $(x_4 \vee \bar{x}_1)$ diverrebbe non soddisfatta e l'algoritmo potrebbe cambiare x_1 o x_4. E così via...

Nel seguente Lemma 7.1 studiamo il numero atteso di iterazioni dell'algoritmo se sappiamo a priori che esiste una soluzione. Nel Teorema 7.2 daremo l'analisi dell'algoritmo nel caso generale.

Lemma 7.1

Assumiamo che un problema 2-SAT con n variabili abbia almeno una soluzione e che l'algoritmo sia lasciato correre finché

non trovi una soluzione. Allora il numero atteso di passi è al più n^2.

Dim.

Sia S una soluzione specifica del problema e chiamiamo A_i l'assegnazione delle variabili all'i-esimo passo dell'algoritmo. Sia $X_i = \#\{\text{variabili in } A_i \text{ che coincidono con quelle di } S\}$

Quando $X_i = n$ l'algoritmo termina avendo trovato la soluzione S (l'algoritmo potrebbe anche terminare prima avendo trovato un'altra soluzione, il che migliorerebbe le cose).

Studiamo quanto ci mette $(X_i)_{i \in \mathbb{N}_0}$ a raggiungere n.

- Se $X_i = 0$, allora qualsiasi cambio di variabili porta ad avere $X_{i+1} = 1$:

$$\mathbb{P}(X_{i+1} = 1 \mid X_i = 0) = 1.$$

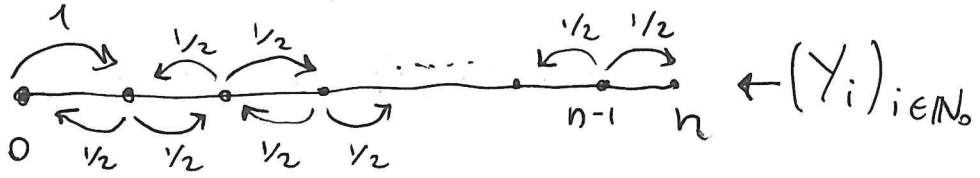
- Se $1 \leq X_i \leq n-1$ si ha quanto segue:

ad ogni passo viene scelta una clausola non soddisfatta. Siccome S è una soluzione, A_i ed S discordano in almeno una delle due variabili della clausola. Perciò

$$\mathbb{P}(X_{i+1} = j+1 \mid X_i = j) \geq \tfrac{1}{2} \qquad \forall j = 1, \dots, n-1$$
$$\mathbb{P}(X_{i+1} = j-1 \mid X_i = j) \leq \tfrac{1}{2}$$

Attenzione! Il processo $X_0, X_1, X_2 \dots$ NON è una catena di Markov (il valore al passo successivo di X_i può dipendere dall'intero passato, pensateci...). Però consideriamo la catena di Markov accessoria $(Y_i)_{i \in \mathbb{N}_0}$ tale che

$$Y_0 = X_0$$
$$\mathbb{P}(Y_{i+1} = 1 \mid Y_i = 0) = 1$$
$$\mathbb{P}(Y_{i+1} = j+1 \mid Y_i = j) = \tfrac{1}{2}$$
$$\mathbb{P}(Y_{i+1} = j-1 \mid Y_i = j) = \tfrac{1}{2}$$



$(Y_i)_{i \in \mathbb{N}_0}$ è una versione "pessimistica" di $(X_i)_{i \in \mathbb{N}_0}$

perché ad ogni passo ha meno probabilità di (X_i) di avanzare verso destra (dicesi: dominazione stocastica, oppure coupling. Vedi capitolo 12 del libro). Quindi $(Y_i)_{i \in \mathbb{N}_0}$ raggiungerà n più lentamente di $(X_i)_{i \in \mathbb{N}_0}$. Usiamo quindi $(Y_i)_{i \in \mathbb{N}_0}$ per un upper bound del # atteso di passi per raggiungere n.

Sia Z_j la v.a.

Z_j = #"passi che $(Y_i)_{i \in \mathbb{N}_0}$ impiega per raggiungere n partendo da j"

e sia $h_j := \mathbb{E}[Z_j]$.

Se $(Y_i)_{\mathbb{N}_0}$ parte da j, al prossimo passo sarà in $j-1$ con proba $\frac{1}{2}$ e in $j+1$ con proba $\frac{1}{2}$

~~Ricorsione~~ Abbiamo perciò

$$\mathbb{E}[Z_j] = \mathbb{E}\left[\frac{1}{2}(1+Z_{j-1}) + \frac{1}{2}(1+Z_{j+1})\right]$$

(il passo in più per spostarsi)

ovvero

$$h_j = \frac{h_{j-1}+1}{2} + \frac{h_{j+1}+1}{2}.$$

Abbiamo quindi il sistema di equazioni

$$\begin{cases} h_n = 0 \\ h_j = \frac{h_{j-1}}{2} + \frac{h_{j+1}}{2} + 1 & \text{per } 1 \le j \le n-1 \\ h_0 = h_1 + 1 \end{cases}$$

L'unica soluzione (abbiamo $n+1$ equazioni con $n+1$ incognite) è $h_j = n^2 - j^2$ (verificatelo!).

Perciò

$\mathbb{E}[\#\text{passi algoritmo per trovare la soluzione } S]$

$\le \mathbb{E}[\#\text{passi per } (Y_i)_{i \in \mathbb{N}_0} \text{ per raggiungere } n$ partendo da $0]$

$= h_0$

$= n^2.$ □

Torniamo al problema generale, in cui non sappiamo se esiste una soluzione e facciamo eseguire il loop all'algoritmo $2mn^2$ volte!

Teorema 7.2

L'algoritmo 2-SAT restituisce sempre fallimento se il problema 2-SAT non ha soluzioni. Se esistono soluzioni,

l'algoritmo ne restituisce una con probabilità maggiore di $1 - 2^{-m}$. Altrimenti, restituisce erroneamente un fallimento.

Dim.

La prima parte del teorema è ovvia. Supponiamo invece esista una soluzione. Dividiamo l'esecuzione dell'algoritmo in blocchi da $2n^2$ passi ciascuno. Se non è stata trovata una soluzione durante i primi $i-1$ blocchi, qual è la probabilità che una soluzione NON venga trovata neanche nell'i-esimo blocco? Dal lemma 7.1 sappiamo che il numero atteso di passi per trovare una soluzione è meno di n^2, indipendentemente dal punto d'inizio. Se chiamiamo

$Z = \#$ "passi dall'inizio dell'i-esimo blocco prima che l'algoritmo trovi una soluzione"

abbiamo che

$$P(Z > 2n^2) \leq \frac{E[Z]}{2n^2} \leq \frac{1}{2}.$$

↑ disuguaglianza di Markov

Quindi la proba che l'algoritmo NON trovi una soluzione dopo m blocchi è minore di $\left(\frac{1}{2}\right)^m$.

□

Oss. La sezione 7.1.2 del libro tratta il problema 3-SAT e descrive un algoritmo del tutto analogo al precedente per trovare una soluzione. È interessante notare che, il problema 3-SAT è NP-completo ed infatti l'algoritmo proposto impiega un numero esponenziale di passi $\left(O(n^{3k}(\frac{4}{3})^n)\right)$ per trovare una soluzione, mentre per il 2-SAT bastava un numero polinomiale $O(n^2)$ di passi!